TOPS Technologies

Secure and Monitor a Cloud Infrastructure

Assessment A6 — Complete Answers

Theme: Cloud Computing

Total Time: 4 Hours

Assessment 6

Section A — Concept Application

Question 1: Cloud service models and shared responsibility

Answer:

Windows Server with Active Directory: AWS Managed Microsoft AD is a suitable managed replacement if the team needs a managed Microsoft directory. It is a managed service rather than a simple IaaS VM. AWS manages the underlying service infrastructure and maintenance; the company still manages directory users, groups, permissions, and identity lifecycle. If custom domain-controller control is essential, Windows Server on EC2 is IaaS, and the team must manage the guest OS, patching, and directory configuration.

Linux customer-facing API: A managed application platform such as AWS Elastic Beanstalk is a PaaS-style option when the API fits its supported runtime. AWS manages much of the platform provisioning and runtime maintenance; the team still secures application code, dependencies, authentication, data, and configuration. Running the API on EC2 is IaaS and leaves more OS administration to the team. SaaS is not the best direct replacement for a custom API because SaaS is a finished application consumed by end users.

Shared responsibility means AWS secures the cloud infrastructure, while the customer secures identities, data, application code, access rules, and service configuration. The exact division depends on the selected AWS service.

Question 2: VPC design for evening order spikes

Answer:

Use a VPC spanning at least two Availability Zones. Place an internet-facing Application Load Balancer (ALB) in public subnets, order-processing EC2 instances/containers in private application subnets, and the database in isolated private database subnets. The database subnet has no Internet Gateway route and the database has no public IP. Use a NAT Gateway only when private workloads need outbound internet access for updates.

Use an ALB because it supports HTTP/HTTPS routing, TLS termination, target health checks, and distribution across Availability Zones. Put the application tier in an Auto Scaling group. Scale on average CPU utilization (for example, target 60%) and/or ALB requests per target; scheduled scaling before the predictable 7–9 PM peak can supplement reactive scaling.

Security groups: ALB inbound TCP 443 from clients; ALB outbound to the application security group on the API port. Application security group inbound only from the ALB security group; outbound only to required dependencies. Database security group inbound only from the

application security group on the database port (for example, 3306 for MySQL or 5432 for PostgreSQL). Do not allow public database ingress.

Question 3: S3 storage for photos and seven-year receipts

Answer:

Store frequently viewed menu photos in S3 Standard. Do not configure a transition or expiration for the photos/ prefix, so completed photo objects remain in Standard unless another lifecycle rule or explicit storage-class change is applied. S3 lifecycle configuration cannot express an explicit 'force Standard forever' action; the correct configuration is to avoid transitions/expiration for photos/.

Store receipts in S3 and transition objects under receipts/ to S3 Glacier Flexible Retrieval after 90 days. Keep the seven-year retention requirement in mind: do not expire receipts before the required retention period. Consider S3 Object Lock if the legal retention requirement requires immutable records, and verify retrieval-time and restore-cost requirements.

EBS is block storage intended for volumes attached to compute instances. It is not ideal for shared, scalable object access or native object lifecycle transitions. S3 is designed for durable objects, bucket-level policies, and lifecycle management.

Question 4: Least-privilege IAM for a backend developer

Answer:

Use a federated identity/IAM Identity Center assignment and a role with a custom read-only policy. Grant only the necessary CloudWatch Logs read actions on the relevant log groups and s3:ListBucket with a receipts/ prefix condition plus s3:GetObject on the required object prefix. Do not grant IAM changes, s3:PutObject, s3:DeleteObject, EC2 termination, or resource deletion. AWS-managed read-only policies can be starting points, but review their scope; a custom policy is more precise for this requirement.

AdministratorAccess would let the developer modify IAM policies, change security settings, delete or alter resources, and potentially expose or destroy production data. It violates least privilege and increases the impact of compromised credentials or accidental commands. Use MFA, temporary credentials, and periodic access reviews.

Question 5: Customer-managed KMS key for Lambda environment variables

Answer:

Create a symmetric customer-managed AWS KMS key and enable automatic key rotation. KMS rotation changes the backing key material while the key ID/ARN remains stable; KMS retains prior key material so previously encrypted data can still be decrypted. Rotation does not rotate the actual payment-provider secret, so that secret must also have its own rotation process.

A customer-managed key provides control over key policy, IAM permissions, grants, rotation settings, disablement, and audit visibility through CloudTrail. An AWS-managed key offers less customer control over key policy and lifecycle. For payment secrets, AWS Secrets Manager or Systems Manager Parameter Store SecureString may be preferable to storing long-lived secrets only in Lambda environment variables.

Question 6: Monitoring and auditing

Answer:

Enable AWS CloudTrail management events and deliver logs to a protected CloudWatch Logs group and/or a dedicated S3 archive. Use AWS Config rules to detect noncompliant IAM and S3 public-access configurations; IAM Access Analyzer to identify unintended external access; and S3 Block Public Access at account and bucket level.

Use EventBridge rules or CloudWatch Logs metric filters for IAM policy/attachment changes and S3 public-access changes, then route alerts through SNS. Use AWS Config managed rules for public S3 access and risky configuration. GuardDuty can detect threats, and Security Hub can aggregate security findings. Test the alerts and restrict access to audit logs.

Section B — Practical Coding Tasks

Task 1: VPC Setup for the Order-Processing Service

```
#!/usr/bin/env bash

# M8-A1 Section B - Task 1: VPC setup

set -euo pipefail



REGION="${AWS_REGION:-ap-south-1}"

VPC_CIDR="${VPC_CIDR:-10.20.0.0/16}"

PUBLIC_CIDR="${PUBLIC_CIDR:-10.20.1.0/24}"

PRIVATE_CIDR="${PRIVATE_CIDR:-10.20.2.0/24}"

AZ1="${AZ1:-${REGION}a}"

AZ2="${AZ2:-${REGION}b}"
```

```
VPC_ID=$(aws ec2 create-vpc --region "$REGION" \
  --cidr-block "$VPC_CIDR" \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=food-delivery-vpc}]' \
  --query 'Vpc.VpcId' --output text)

aws ec2 modify-vpc-attribute --region "$REGION" --vpc-id "$VPC_ID" --enable-dns-support '{"Value":true}'
aws ec2 modify-vpc-attribute --region "$REGION" --vpc-id "$VPC_ID" --enable-dns-hostnames '{"Value":true}'

PUBLIC_SUBNET_ID=$(aws ec2 create-subnet --region "$REGION" \
  --vpc-id "$VPC_ID" --cidr-block "$PUBLIC_CIDR" --availability-zone "$AZ1" \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=food-delivery-public}]' \
  --query 'Subnet.SubnetId' --output text)

PRIVATE_SUBNET_ID=$(aws ec2 create-subnet --region "$REGION" \
  --vpc-id "$VPC_ID" --cidr-block "$PRIVATE_CIDR" --availability-zone "$AZ2" \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=food-delivery-private}]' \
  --query 'Subnet.SubnetId' --output text)
```

```
IGW_ID=$(aws ec2 create-internet-gateway --region "$REGION" \

  --tag-specifications
'ResourceType=internet-gateway,Tags=[{Key=Name,Value=food-delivery-igw}]' \

  --query 'InternetGateway.InternetGatewayId' --output text)

aws ec2 attach-internet-gateway --region "$REGION" --internet-gateway-id "$IGW_ID"
--vpc-id "$VPC_ID"



RT_ID=$(aws ec2 create-route-table --region "$REGION" --vpc-id "$VPC_ID" \

  --tag-specifications
'ResourceType=route-table,Tags=[{Key=Name,Value=food-delivery-public-rt}]' \

  --query 'RouteTable.RouteTableId' --output text)

aws ec2 create-route --region "$REGION" --route-table-id "$RT_ID" \

  --destination-cidr-block 0.0.0.0/0 --gateway-id "$IGW_ID" >/dev/null

aws ec2 associate-route-table --region "$REGION" --route-table-id "$RT_ID" \

  --subnet-id "$PUBLIC_SUBNET_ID" >/dev/null



# No internet route is created for the private subnet.

echo "VPC_ID=$VPC_ID"

echo "PUBLIC_SUBNET_ID=$PUBLIC_SUBNET_ID"

echo "PRIVATE_SUBNET_ID=$PRIVATE_SUBNET_ID"

echo "INTERNET_GATEWAY_ID=$IGW_ID"
```

```
echo "PUBLIC_ROUTE_TABLE_ID=$RT_ID"

echo "Region: $REGION"
```

Task 2: Tiered Storage for Menu Photos and Receipts

```
#!/usr/bin/env bash

# M8-A1 Section B - Task 2: S3 lifecycle rules

set -euo pipefail

REGION="${AWS_REGION:-ap-south-1}"

# S3 bucket names must be globally unique. Override this value before running.

BUCKET="${BUCKET_NAME:-food-delivery-m8a1-change-me-20261028}"

LIFECYCLE_FILE="$(mktemp)"

trap 'rm -f "$LIFECYCLE_FILE"' EXIT



if [[ "$BUCKET" == *change-me* ]]; then

  echo "ERROR: Set BUCKET_NAME to a globally unique bucket name before running." >&2

  exit 1

fi



if [[ "$REGION" == "us-east-1" ]]; then

  aws s3api create-bucket --region "$REGION" --bucket "$BUCKET"

else

  aws s3api create-bucket --region "$REGION" --bucket "$BUCKET" \
```

```
    --create-bucket-configuration "LocationConstraint=$REGION"
fi

aws s3api put-public-access-block --region "$REGION" --bucket "$BUCKET" \
  --public-access-block-configuration \
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tru
e

aws s3api put-bucket-encryption --region "$REGION" --bucket "$BUCKET" \
  --server-side-encryption-configuration \
'{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"},"BucketKey
Enabled":true}]}'

cat > "$LIFECYCLE_FILE" <<'JSON'
{
  "Rules": [
    {
      "ID": "Receipts-Transition-To-Glacier-After-90-Days",
      "Status": "Enabled",
      "Filter": {"Prefix": "receipts/"},
```

```
      "Transitions": [{"Days": 90, "StorageClass": "GLACIER"}]
    },
    {
      "ID": "Photos-Remain-In-Standard",
      "Status": "Enabled",
      "Filter": {"Prefix": "photos/"},
      "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
    }
  ]
}
JSON

aws s3api put-bucket-lifecycle-configuration --region "$REGION" \
  --bucket "$BUCKET" --lifecycle-configuration "file://$LIFECYCLE_FILE"

echo "Lifecycle rules applied. Confirming configuration:"
aws s3api get-bucket-lifecycle-configuration --region "$REGION" --bucket "$BUCKET"
echo "BUCKET_NAME=$BUCKET"
```

Task 3: Least-Privilege Role for the Delivery-Tracking Service

```
#!/usr/bin/env bash
```

```
# M8-A1 | Section B | Task 3
set -euo pipefail

REGION="${AWS_REGION:-ap-south-1}"
BUCKET="${BUCKET_NAME:-food-delivery-m8a1-change-me-20261028}"
ROLE_NAME="${ROLE_NAME:-DeliveryTrackingRole}"
TRUST_FILE="$(mktemp)"
POLICY_FILE="$(mktemp)"
trap 'rm -f "$TRUST_FILE" "$POLICY_FILE"' EXIT

if [[ "$BUCKET" == *change-me* ]]; then
  echo "ERROR: Set BUCKET_NAME to the bucket created in Section B Task 2." >&2
  exit 1
fi

cat > "$TRUST_FILE" <<'JSON'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
```

```
    "Principal": {"Service": "ec2.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
JSON

cat > "$POLICY_FILE" <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::$BUCKET/*"
  }]
}
JSON

aws iam create-role \
  --role-name "$ROLE_NAME" \
  --assume-role-policy-document "file://$TRUST_FILE" \
```

```
  --description "Least-privilege role for delivery-tracking service"


aws iam put-role-policy \
  --role-name "$ROLE_NAME" \
  --policy-name "DeliveryTrackingObjectAccess" \
  --policy-document "file://$POLICY_FILE"


echo "Role ARN:"
aws iam get-role --role-name "$ROLE_NAME" --query 'Role.Arn' --output text
echo "Attached inline policy:"
aws iam get-role-policy --role-name "$ROLE_NAME" \
  --policy-name "DeliveryTrackingObjectAccess"
```

Task 4: Secure and Monitored Order Database Backup

```
#!/usr/bin/env bash

# M8-A1 Section B - Task 4: Encrypted backup bucket and alerting

# Prerequisite: a CloudTrail trail must deliver management events to the specified CloudWatch Logs group.

set -euo pipefail

REGION="${AWS_REGION:-ap-south-1}"
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET="${BACKUP_BUCKET_NAME:-food-delivery-backups-change-me-20261028}"
```

```
ROLE_NAME="${BACKUP_ROLE_NAME:-OrderBackupWriterRole}"

LOG_GROUP="${CLOUDTRAIL_LOG_GROUP:-/aws/cloudtrail/food-delivery}"

TOPIC_NAME="${SNS_TOPIC_NAME:-FoodDeliverySecurityAlerts}"

EMAIL="${ALERT_EMAIL:-}"

KEY_ALIAS="${KMS_ALIAS:-alias/food-delivery-backups}"

TMPDIR_TASK4="$(mktemp -d)"

trap 'rm -rf "$TMPDIR_TASK4"' EXIT



if [[ "$BUCKET" == *change-me* ]]; then

  echo "ERROR: Set BACKUP_BUCKET_NAME to a globally unique bucket name." >&2; exit
1

fi

if [[ -z "$EMAIL" ]]; then

  echo "ERROR: Set ALERT_EMAIL to an email address for SNS notifications." >&2; exit 1

fi



KEY_ID=$(aws kms create-key --region "$REGION" \

  --description "Customer-managed key for food delivery database backups" \

  --key-usage ENCRYPT_DECRYPT --origin AWS_KMS \

  --query 'KeyMetadata.KeyId' --output text)

aws kms enable-key-rotation --region "$REGION" --key-id "$KEY_ID"
```

```
aws kms create-alias --region "$REGION" --alias-name "$KEY_ALIAS" --target-key-id "$KEY_ID" || true



if [[ "$REGION" == "us-east-1" ]]; then

  aws s3api create-bucket --region "$REGION" --bucket "$BUCKET"

else

  aws s3api create-bucket --region "$REGION" --bucket "$BUCKET" \

    --create-bucket-configuration "LocationConstraint=$REGION"

fi

aws s3api put-public-access-block --region "$REGION" --bucket "$BUCKET" \

  --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

aws s3api put-bucket-encryption --region "$REGION" --bucket "$BUCKET" \

  --server-side-encryption-configuration "{\"Rules\":[{\"ApplyServerSideEncryptionByDefault\":{\"SSEAlgorithm\":\"aws:kms\",\"KMSMasterKeyID\":\"$KEY_ID\"},\"BucketKeyEnabled\":true}]}"



cat > "$TMPDIR_TASK4/trust.json" <<'JSON'

{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}

JSON

cat > "$TMPDIR_TASK4/policy.json" <<JSON
```

```
{

 "Version":"2012-10-17",

 "Statement":[

  {"Effect":"Allow","Action":["s3:PutObject"],"Resource":"arn:aws:s3:::$BUCKET/*"},


{"Effect":"Allow","Action":["kms:Encrypt","kms:GenerateDataKey","kms:DescribeKey"],"Res
ource":"arn:aws:kms:$REGION:$ACCOUNT_ID:key/$KEY_ID"}

 ]

}
```

JSON

```
aws iam create-role --role-name "$ROLE_NAME" --assume-role-policy-document
"file://$TMPDIR_TASK4/trust.json" \

  --description "Writes encrypted database backups to the dedicated backup bucket"

aws iam put-role-policy --role-name "$ROLE_NAME" --policy-name
"EncryptedBackupWriteOnly" \

  --policy-document "file://$TMPDIR_TASK4/policy.json"



TOPIC_ARN=$(aws sns create-topic --region "$REGION" --name "$TOPIC_NAME" --query
TopicArn --output text)

SUB_ARN=$(aws sns subscribe --region "$REGION" --topic-arn "$TOPIC_ARN" \

  --protocol email --notification-endpoint "$EMAIL" --query SubscriptionArn --output text)
```

```
# CloudTrail event history must be delivered to this log group for the metric filter to work.

# Filter tracks successful S3 PutPublicAccessBlock API calls; treat any such change as requiring investigation.

FILTER_NAME="S3PublicAccessBlockChanged"

aws logs put-metric-filter --region "$REGION" --log-group-name "$LOG_GROUP" \

  --filter-name "$FILTER_NAME" \

  --filter-pattern '{ ($.eventSource = "s3.amazonaws.com") && ($.eventName = "PutPublicAccessBlock") }' \

  --metric-transformations metricName=S3PublicAccessBlockChange,metricNamespace=FoodDelivery/Security,metricValue=1,defaultValue=0



aws cloudwatch put-metric-alarm --region "$REGION" --alarm-name "Alert-S3-PublicAccessBlock-Changed" \

  --alarm-description "Alert when an S3 bucket public access block configuration is changed" \

  --namespace "FoodDelivery/Security" --metric-name "S3PublicAccessBlockChange" \

  --statistic Sum --period 300 --evaluation-periods 1 --threshold 1 \

  --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data notBreaching \

  --alarm-actions "$TOPIC_ARN"



echo "Created resources summary"

echo "REGION=$REGION"

echo "KMS_KEY_ID=$KEY_ID"
```

```
echo "BACKUP_BUCKET=$BUCKET"

echo "IAM_ROLE_ARN=$(aws iam get-role --role-name "$ROLE_NAME" --query 'Role.Arn' --output text)"

echo "CLOUDWATCH_LOG_GROUP=$LOG_GROUP"

echo "METRIC_FILTER=$FILTER_NAME"

echo "CLOUDWATCH_ALARM=Alert-S3-PublicAccessBlock-Changed"

echo "SNS_TOPIC_ARN=$TOPIC_ARN"

echo "SNS_SUBSCRIPTION_ARN=$SUB_ARN (email subscriptions require confirmation)"

echo "IMPORTANT: Ensure the CloudTrail trail is actively delivering events to $LOG_GROUP."
```

Section C — Mini Capstone Project

Mini Project: Secure & Monitored Cloud Infrastructure for a Food Delivery Platform

```
#!/usr/bin/env bash

# M8-A1 Section C - Menu-driven secure cloud infrastructure capstone.

# Creates core networking, two scoped roles, KMS-encrypted S3 buckets and monitoring.

# Prerequisite: AWS CLI v2 configured; jq installed; a CloudTrail log group exists.

set -euo pipefail

REGION="${AWS_REGION:-ap-south-1}"

SUFFIX="${RESOURCE_SUFFIX:-$(date -u +%Y%m%d)-$RANDOM}"

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

VPC_CIDR="${VPC_CIDR:-10.30.0.0/16}"
```

```
PUBLIC_CIDR="${PUBLIC_CIDR:-10.30.1.0/24}"

PRIVATE_CIDR="${PRIVATE_CIDR:-10.30.2.0/24}"

BUCKET_APP="food-delivery-app-${ACCOUNT_ID}-${SUFFIX}"

BUCKET_BACKUP="food-delivery-backup-${ACCOUNT_ID}-${SUFFIX}"

LOG_GROUP="${CLOUDTRAIL_LOG_GROUP:-/aws/cloudtrail/food-delivery}"

EMAIL="${ALERT_EMAIL:-}"

SUMMARY_FILE="${SUMMARY_FILE:-./m8a1_resource_summary.txt}"

KEY_ID=""

VPC_ID=""

PUB_ID=""

PRIV_ID=""

IGW_ID=""

RT_ID=""

APP_ROLE_ARN=""

TRACK_ROLE_ARN=""

TOPIC_ARN=""

ALARM_NAME="FoodDelivery-PublicAccessBlock-Changed-${SUFFIX}"



record(){ printf '%s\n' "$*" | tee -a "$SUMMARY_FILE"; }

init_summary(){ printf 'M8-A1 Resource Summary\nRegion: %s\nSuffix: %s\n' "$REGION"
"$SUFFIX" > "$SUMMARY_FILE"; }
```

```
provision_network(){

  if [[ -n "$VPC_ID" ]]; then echo "Networking already created in this run: $VPC_ID"; return; fi

  VPC_ID=$(aws ec2 create-vpc --region "$REGION" --cidr-block "$VPC_CIDR" \

    --tag-specifications
"ResourceType=vpc,Tags=[{Key=Name,Value=FoodDelivery-${SUFFIX}}]" \

    --query Vpc.VpcId --output text)

  aws ec2 modify-vpc-attribute --region "$REGION" --vpc-id "$VPC_ID" --enable-dns-support
'{"Value":true}'

  aws ec2 modify-vpc-attribute --region "$REGION" --vpc-id "$VPC_ID"
--enable-dns-hostnames '{"Value":true}'

  PUB_ID=$(aws ec2 create-subnet --region "$REGION" --vpc-id "$VPC_ID" --cidr-block
"$PUBLIC_CIDR" \

    --availability-zone "${REGION}a" --tag-specifications
"ResourceType=subnet,Tags=[{Key=Name,Value=FoodDelivery-Public-${SUFFIX}}]" --query
Subnet.SubnetId --output text)

  PRIV_ID=$(aws ec2 create-subnet --region "$REGION" --vpc-id "$VPC_ID" --cidr-block
"$PRIVATE_CIDR" \

    --availability-zone "${REGION}b" --tag-specifications
"ResourceType=subnet,Tags=[{Key=Name,Value=FoodDelivery-Private-${SUFFIX}}]" --query
Subnet.SubnetId --output text)

  IGW_ID=$(aws ec2 create-internet-gateway --region "$REGION" --query
InternetGateway.InternetGatewayId --output text)

  aws ec2 attach-internet-gateway --region "$REGION" --vpc-id "$VPC_ID"
--internet-gateway-id "$IGW_ID"
```

```
  RT_ID=$(aws ec2 create-route-table --region "$REGION" --vpc-id "$VPC_ID" --query
RouteTable.RouteTableId --output text)

  aws ec2 create-route --region "$REGION" --route-table-id "$RT_ID" --destination-cidr-block
0.0.0.0/0 --gateway-id "$IGW_ID" >/dev/null

  aws ec2 associate-route-table --region "$REGION" --route-table-id "$RT_ID" --subnet-id
"$PUB_ID" >/dev/null

  record "VPC_ID=$VPC_ID"; record "PUBLIC_SUBNET_ID=$PUB_ID"; record
"PRIVATE_SUBNET_ID=$PRIV_ID"

  record "INTERNET_GATEWAY_ID=$IGW_ID"; record
"PUBLIC_ROUTE_TABLE_ID=$RT_ID"

  record "Private subnet has no direct internet route. Add a NAT gateway only if outbound
internet is required."

}

secure_storage_and_iam(){

  if [[ -z "$KEY_ID" ]]; then

    KEY_ID=$(aws kms create-key --region "$REGION" --description "Food delivery capstone
CMK ${SUFFIX}" --query KeyMetadata.KeyId --output text)

    aws kms enable-key-rotation --region "$REGION" --key-id "$KEY_ID"

    record "KMS_KEY_ID=$KEY_ID (automatic rotation enabled)"

  fi

  for B in "$BUCKET_APP" "$BUCKET_BACKUP"; do

    if [[ "$REGION" == "us-east-1" ]]; then aws s3api create-bucket --region "$REGION"
--bucket "$B"

    else aws s3api create-bucket --region "$REGION" --bucket "$B"
--create-bucket-configuration "LocationConstraint=$REGION"; fi
```

```
    aws s3api put-public-access-block --region "$REGION" --bucket "$B"
--public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=tru
e

    aws s3api put-bucket-encryption --region "$REGION" --bucket "$B"
--server-side-encryption-configuration
"{\"Rules\":[{\"ApplyServerSideEncryptionByDefault\":{\"SSEAlgorithm\":\"aws:kms\",\"KMS
MasterKeyID\":\"$KEY_ID\"},\"BucketKeyEnabled\":true}]}"

    record "S3_BUCKET=$B (public access blocked; SSE-KMS enabled)"

  done

  # Two separate EC2 service roles, with different resource scopes.

  for R in "OrderProcessingRole" "DeliveryTrackingRole"; do

    cat > "/tmp/${R}-${SUFFIX}-trust.json" <<'JSON'

{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":{"Service":"ec2.amazonaw
s.com"},"Action":"sts:AssumeRole"}]}

JSON

    aws iam create-role --role-name "${R}-${SUFFIX}" --assume-role-policy-document
"file:///tmp/${R}-${SUFFIX}-trust.json" >/dev/null

  done

  cat > "/tmp/${SUFFIX}-order-policy.json" <<JSON

{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["s3:GetObject","s3:PutObje
ct"],"Resource":"arn:aws:s3:::$BUCKET_APP/*"},{"Effect":"Allow","Action":["kms:Encrypt",
"kms:GenerateDataKey","kms:Decrypt","kms:DescribeKey"],"Resource":"arn:aws:kms:$REGI
ON:$ACCOUNT_ID:key/$KEY_ID"}]}

JSON
```

```
  cat > "/tmp/${SUFFIX}-tracking-policy.json" <<JSON

{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["s3:GetObject","s3:PutObje
ct"],"Resource":"arn:aws:s3:::$BUCKET_BACKUP/*"},{"Effect":"Allow","Action":["kms:Encr
ypt","kms:GenerateDataKey","kms:Decrypt","kms:DescribeKey"],"Resource":"arn:aws:kms:$R
EGION:$ACCOUNT_ID:key/$KEY_ID"}]}

JSON

  aws iam put-role-policy --role-name "OrderProcessingRole-${SUFFIX}" --policy-name
"OrderProcessingScopedAccess" --policy-document "file:///tmp/${SUFFIX}-order-policy.json"

  aws iam put-role-policy --role-name "DeliveryTrackingRole-${SUFFIX}" --policy-name
"DeliveryTrackingScopedAccess" --policy-document
"file:///tmp/${SUFFIX}-tracking-policy.json"

  APP_ROLE_ARN=$(aws iam get-role --role-name "OrderProcessingRole-${SUFFIX}"
--query Role.Arn --output text)

  TRACK_ROLE_ARN=$(aws iam get-role --role-name "DeliveryTrackingRole-${SUFFIX}"
--query Role.Arn --output text)

  record "ORDER_PROCESSING_ROLE_ARN=$APP_ROLE_ARN"; record
"DELIVERY_TRACKING_ROLE_ARN=$TRACK_ROLE_ARN"

  rm -f "/tmp/OrderProcessingRole-${SUFFIX}-trust.json"
"/tmp/DeliveryTrackingRole-${SUFFIX}-trust.json" "/tmp/${SUFFIX}-order-policy.json"
"/tmp/${SUFFIX}-tracking-policy.json"

}

enable_monitoring(){

  if [[ -z "$EMAIL" ]]; then read -r -p "Enter email for alarm notifications: " EMAIL; fi

  TOPIC_ARN=$(aws sns create-topic --region "$REGION" --name
"FoodDeliveryAlerts-${SUFFIX}" --query TopicArn --output text)

  aws sns subscribe --region "$REGION" --topic-arn "$TOPIC_ARN" --protocol email
--notification-endpoint "$EMAIL" >/dev/null
```

```
  aws logs put-metric-filter --region "$REGION" --log-group-name "$LOG_GROUP"
--filter-name "S3PublicAccessBlockChanged-${SUFFIX}" \

    --filter-pattern '{ ($.eventSource = "s3.amazonaws.com") && ($.eventName =
"PutPublicAccessBlock") }' \

    --metric-transformations
"metricName=S3PublicAccessBlockChange,metricNamespace=FoodDelivery/Security,metricVa
lue=1,defaultValue=0"

  aws cloudwatch put-metric-alarm --region "$REGION" --alarm-name "$ALARM_NAME" \

    --alarm-description "S3 public access block setting changed; investigate" \

    --namespace FoodDelivery/Security --metric-name S3PublicAccessBlockChange --statistic
Sum \

    --period 300 --evaluation-periods 1 --threshold 1 --comparison-operator
GreaterThanOrEqualToThreshold \

    --treat-missing-data notBreaching --alarm-actions "$TOPIC_ARN"

  record "SNS_TOPIC_ARN=$TOPIC_ARN"; record
"CLOUDWATCH_ALARM=$ALARM_NAME"

  record "SNS email subscription requires confirmation. CloudTrail must deliver events to
$LOG_GROUP."

}

view_summary(){ echo "===== Resource Summary ====="; cat "$SUMMARY_FILE"; }

init_summary

while true; do

  echo

  echo "M8-A1 Secure Cloud Infrastructure"
```

```
  select choice in "Provision Infrastructure" "Apply Security & Encryption" "Enable Monitoring & Alerts" "View Resource Summary" "Exit"; do

    case "$REPLY" in

      1) provision_network; break ;;

      2) secure_storage_and_iam; break ;;

      3) enable_monitoring; break ;;

      4) view_summary; break ;;

      5) echo "Exiting. Summary saved to $SUMMARY_FILE"; exit 0 ;;

      *) echo "Choose a number from 1 to 5." ;;

    esac

  done

done
```

Section D — AI-Augmented Learning

Step 1: Build with AI

```
#!/usr/bin/env bash

# M8-A1 | Section D | AI-generated baseline (save this as the original version)

set -euo pipefail

REGION="${AWS_REGION:-ap-south-1}"

BUCKET="${RECEIPTS_BUCKET:-food-delivery-receipts-change-me-20261028}"

NOW=$(date -u +%s)

TOKEN=""
```

```
printf "%-65s %-10s %-15s\n" "FILENAME" "AGE_DAYS" "STORAGE_CLASS"
printf '%s\n' "-----------------------------------------------------------------------------------------"

while :; do
  if [[ -n "$TOKEN" ]]; then
    RESPONSE=$(aws s3api list-objects-v2 --region "$REGION" --bucket "$BUCKET" \
      --prefix "receipts/" --continuation-token "$TOKEN" --output json)
  else
    RESPONSE=$(aws s3api list-objects-v2 --region "$REGION" --bucket "$BUCKET" \
      --prefix "receipts/" --output json)
  fi

  echo "$RESPONSE" | jq -r --argjson now "$NOW" '
    (.Contents // [])[]
    | . as $o
    | (($now - ($o.LastModified | fromdateiso8601)) / 86400 | floor) as $age
    | ($o.StorageClass // "STANDARD") as $class
    | select($age > 90 and $class == "STANDARD")
    | [$o.Key, ($age|tostring), $class] | @tsv
```

```
  ' | while IFS=$'\t' read -r key age class; do
    printf "%-65s %-10s %-15s\n" "$key" "$age" "$class"
  done

  [[ "$(echo "$RESPONSE" | jq -r '.IsTruncated // false')" == "true" ]] || break
  TOKEN=$(echo "$RESPONSE" | jq -r '.NextContinuationToken // empty')
done
```

Step 2: Test and Debug Without AI